

SOMMAIRE

1.	Perimetre de l'analyse	2
2.	Quand l'auto vacuum se déclenche-t-il ?	2
3.	Mode de fonctionnement de auto vacuum launcher.....	2
4.	Quels sont les objets éligibles à l'auto Vacuum ?	3
4.1	Identification des paramètres système	3
4.2	Récupération des statistiques sur les tables.....	4
4.3	Récupération de l'âge des objets	5
4.4	Seuil d'éligibilité a l'autovacuum d'un objet.....	6
4.4.1	Seuil d'éligibilité a l'auto VACUUM d'un objet sur les évènements (UPDATE ou DELETE).....	6
4.4.2	Seuil d'éligibilité a l'auto VACUUM d'un objet sur l'évènements (INSERT).....	7
4.4.3	Seuil d'éligibilité a l'auto ANALYSE d'un objet	7
5.	Déclenchement de l'auto VACUUM et ANALYSE	8
6.	Comment forcer (tunning) VACUUM et ANALYSE	9
7.	Pour aller plus loin	10

1. Perimetre de l'analyse

On souhaite comprendre :

- Quand un Auto (Vacuum ou Analyse) se déclenche.
- Comment le « daemon autovacuum » se comporte-il une fois déclenché
- Comment les tables sont éligibles a un Auto (VACUUM, ANALYSE ou FREEZE).
- Comment Configurer individuellement certaines tables pour une Auto (Analyse ou Vacuum)
- Quel Alternative a-ton pour compléter un Auto (Vacuum et Analyse)

Pour une compréhension de ce document il est conseillé de lire le doc suivant :

<https://docs.postgresql.fr/14/routine-vacuuming.html>

2. Quand l'auto vacuum se déclenche-t-il ?

Le démon `autovacuum launcher` s'occupe de lancer des **workers** régulièrement sur les Différentes bases .

Ce processus inspecte les statistiques sur les tables (vue `pg_stat_all_tables`) : nombres de lignes insérées, modifiées et supprimées.

Quand certains seuils sont dépassés sur un objet, le **worker** effectue :

un `VACUUM`,
ou un `VACUUM ANALYZE`
ou un `VACUUM FREEZE`

- Comment les *workers* sont-ils déclenchés ?
 - ✚ En définissant la fréquence des workers
`autovacuum_naptime=1min` (paramètre par défaut)
Toutes les une minutes des workers sont lancés.
 - ✚ En définissant le nombre maximum de worker qui peuvent tourner en parallèle
`autovacuum_max_workers=3` (paramètre par défaut)
un worker traite une et une table à la fois
✓ tuning
on peut augmenter ce paramètre si il y a beaucoup de petites à traiter
en cas d'augmentation de `autovacuum_max_workers` il faut aussi penser à augmenter :
`autovacuum_vacuum_cost_delay=2ms`
`autovacuum_vacuum_cost_limit=-1`
car les workers se les partagent

3. Mode de fonctionnement de auto vacuum launcher

- Le traitement VACUUM se déroule en trois passes :



Passe 1

Cette première passe parcourt la table à nettoyer, à la recherche d'enregistrements morts en fonction des (X_{max} , et X_{min} d'un bloc de donnée).

La session effectuant le vacuum garde en mémoire

`maintenance_work_mem=64Mb` (default) la liste des adresses des enregistrements morts.

A ce stade aucun traitement n'est effectué.



Passe 2

Cette passe se charge de nettoyer les entrées d'index. VACUUM possède une liste de `tid` (tuple id récupérer dans la première passe) à invalider. Il parcourt donc tous les index de la table à la recherche de ces `tid`.

A ce stade les entrées d'index correspondant a des lignes mortes sont réellement supprimés.



Passe 3

Après la suppression des id des index , les blocs morts peuvent réellement être supprimés. Alors ces blocs sont écrit dans la Free Space Map (FSM). **Mais ce espace n'est pas rendu**

✓ Tunnig

Si l'espace mémoire `maintenance_work_mem=64Mb` n'est pas suffisant pour enregistrer tous les (X_{max} , et X_{min} d'un objet) Les trois passes sont repetés plusieurs fois, il faut donc augmenter ce paramètre.

4. Quels sont les objets éligibles à l'auto Vacuum ?

4.1 Identification des paramètres système

- Identifier les paramètres système qui interviennent dans l'éligibilité des objets a l'autovacuum, dans le fichier de la configuration `postgresql.conf`. (dans le cas décrit ces paramètres sont ceux par default sur la base de DEV).

```
select name,setting from pg_settings where name in
('autovacuum_vacuum_threshold','autovacuum_vacuum_scale_factor','autovacuu
```

```
m_vacuum_insert_threshold','autovacuum_vacuum_insert_scale_factor','autova
cuum_analyze_threshold','autovacuum_analyze_scale_factor',
'autovacuum_freeze_max_age');
```

name	setting
autovacuum_analyze_scale_factor	0.1
autovacuum_analyze_threshold	50
autovacuum_freeze_max_age	200000000
autovacuum_vacuum_insert_scale_factor	0.2
autovacuum_vacuum_insert_threshold	1000
autovacuum_vacuum_scale_factor	0.2
autovacuum_vacuum_threshold	50

4.2 Récupération des statistiques sur les tables

- Récupérer les informations sur les statistiques des tables
(cette requête est a titre d'exemple et ne récupère que les 4 premières lignes sur la base de DEV)

```
SELECT
s.relname as "relname",
s.n_dead_tup as "n_dead_tup",
s.n_ins_since_vacuum as "n_ins_since_vacuum",
s.n_mod_since_analyze as "n_mod_since_analyze" ,
s.n_live_tup as "n_live_tup",
c.reltuples as "reltuples"
FROM pg_stat_user_tables s JOIN pg_class c ON s.relname = c.relname
ORDER BY 2 DESC LIMIT 4;
-[ RECORD 1 ]-----+-----
relname          | switch
n_dead_tup       | 7309252
n_ins_since_vacuum | 3611754
n_mod_since_analyze | 1550306
```

```

n_live_tup      | 46722891
reltuples       | 4.6750908e+07
-[ RECORD 2 ]-----+-----
relname        | tapchangerstep
n_dead_tup     | 2182462
n_ins_since_vacuum | 2552437
n_mod_since_analyze | 875718
n_live_tup     | 15070129
reltuples      | 1.5001867e+07
-[ RECORD 3 ]-----+-----
relname        | temporarylimit
n_dead_tup     | 1666115
n_ins_since_vacuum | 439131
n_mod_since_analyze | 637477
n_live_tup     | 11970682
reltuples      | 1.1913083e+07
-[ RECORD 4 ]-----+-----
relname        | permanentlimit
n_dead_tup     | 1246201
n_ins_since_vacuum | 931148
n_mod_since_analyze | 91898
n_live_tup     | 8252819
reltuples      | 8.230249e+06

```

4.3 Récupération de l'âge des objets

- Récupérer les informations sur l'âge (relfrozenxid) ici nommer `xid_age` de la table.
(Ici à titre d'exemple on récupère l'âge des tables sélectionnées en 4.2)

```

SELECT
    relname,
    age(relfrozenxid) as xid_age,
    ROUND(100.0 * age(relfrozenxid) /
current_setting('vacuum_freeze_table_age')::numeric, 1) || '%' AS "% til
vacuum freeze"
FROM
    pg_class
WHERE relkind = 'r'
AND relname in
('switch','tapchangerstep','temporarylimit','permanentlimit') order by 2
DESC;

```

relname	xid_age	% til vacuum freeze
permanentlimit	1483435	1.0%
switch	1483425	1.0%
tapchangerstep	1483423	1.0%
temporarylimit	1483421	1.0%

4.4 Seuil d'éligibilité a l'autovacuum d'un objet

- Calculer les seuils de déclenchements de l'AUTO_VACUUM ou l'AUTO_ANALYSE pour une table.
 - ✚ les paramètres suivants, interviennent dans les règles de calcul du seuil d'éligibilité d'un objet a l'auto vacuum (comme déjà identifier en 4.1).
 - ✚ sont globaux pour toutes les bases , mais peuvent être surchargés pour chaque table en fonction de la spécificité de chacune d'elles

```
AUTOVACUUM_ANALYZE_SCALE_FACTOR  
AUTOVACUUM_ANALYZE_THRESHOLD  
AUTOVACUUM_VACUUM_INSERT_SCALE_FACTOR  
AUTOVACUUM_VACUUM_INSERT_THRESHOLD  
AUTOVACUUM_VACUUM_SCALE_FACTOR  
AUTOVACUUM_VACUUM_THRESHOLD  
AUTOVACUUM_FREEZE_MAX_AGE
```

- la règle de calcul du seuil d'éligibilité a l'auto vacuum d'un objet est la suivante :

$\text{seuil_eligibilte} = (\text{parametre} + (\text{ratio_du_parametre} * \text{nombre_de_ligne_dans_objet}))$

4.4.1 Seuil d'éligibilité a l'auto VACUUM d'un objet sur les évènements (UPDATE ou DELETE)

```
vac_up_del =  
autovacuum_vacuum_threshold + (autovacuum_vacuum_scale_factor * reltuples)
```

Exemple :

Calcule de **vac_up_del** pour la table **switch** mentionné en 4.2 avec :
autovacuum_vacuum_threshold = 50
autovacuum_vacuum_scale_factor = 0.2

```
SELECT (50 + (c.reltuples*0.2)) as "vac_up_del"
FROM pg_stat_user_tables s JOIN pg_class c ON s.relname = c.relname
WHERE s.relname = 'switch';
vac_up_del
-----
9402154
```

4.4.2 Seuil d'éligibilité a l'auto VACUUM d'un objet sur l'évènements (INSERT)

```
vac_ins = autovacuum_vacuum_insert_threshold +
(autovacuum_vacuum_insert_scale_factor * reltuples)
```

Exemple :

Calcule de **vac_ins** pour la table **switch** mentionné en 4.2 avec :
autovacuum_vacuum_insert_threshold = 1000
autovacuum_vacuum_insert_scale_factor = 0.2

```
SELECT (1000 + (0.2 * c.reltuples)) as "vac_ins"
FROM pg_stat_user_tables s JOIN pg_class c ON s.relname = c.relname
WHERE s.relname = 'switch';
vac_ins
-----
9403104
```

4.4.3 Seuil d'éligibilité a l'auto ANALYSE d'un objet

```
analys = autovacuum_analyze_threshold + (autovacuum_analyze_scale_factor *
reltuples)
```

Exemple :

Calcule de **analys** pour la table **switch** mentionné en 4.2 avec :
autovacuum_analyze_threshold = 50
autovacuum_analyze_scale_factor = 0.1

```
SELECT (50 + (0.1 * c.reltuples)) as "analys"
```

```
FROM pg_stat_user_tables s JOIN pg_class c ON s.relname = c.relname
WHERE s.relname = 'switch';
```

analys

4780350

5. Déclenchement de l'auto VACUUM et ANALYSE

On a tous élément pour configurer le déclenchement de l'auto ANALYSE, de l'auto VACUUM et du FREEZE sur un objet.

🚦 Un VACUUM_AUTO se déclenche pour un objet si :

Regle vacuum auto

```
(xid_age > autovacuum_freeze_max_age)
Ou
(n_dead_tup > vac_up_del)
Ou
((autovacuum_vacuum_insert_threshold >= 0) et (n_ins_since_vacuum >
vac_ins))
```

Exemple :

Pour la Table switch avec les résultats trouvés :

xid_age = 1483425	(chp.4.3)
autovacuum_freeze_max_age = 200000000	(Chp. 4.1)
n_dead_tup = 7309252	(Chp. 4.2)
vac_up_del = 9402154	(Chp. 4.4.1)
autovacuum_vacuum_insert_threshold = 1000	(Chp. 4.1)
n_ins_since_vacuum = 3611754	(Chp. 4.2)
vac_ins = 9403104	(Chp. 4.4.2)

en appliquant la regle vacuum_auto avec les valeurs récupéré pour la table switch

```
(1483425 > 200000000) faux
Ou
(7309252 > 9402154) faux
Ou
((1000 >= 0) et (3611754 > 9403104)) faux
```

La table switch n'est donc pas éligible à un vacuum auto

🚦 Un VACUUM_ANALYSE_AUTO se déclenche si:

Règle analyse_auto

```
(n_mod_since_analyze > analys)
```

Exemple :

Pour la Table switch avec les résultats trouvés :

n_mod_since_analyze = 1550306 (Chp. 4.2)

analys = 4780350 (Chp. 4.4.3)

en appliquant la regle vacuum_auto avec les valeurs récupéré pour la table switch

```
(1550306 > 4780350) faux
```

La table switch n'est donc pas éligible à un analyse auto

6. Comment forcer (tunning) VACUUM et ANALYSE

✚ Les paramètres :

```
AUTOVACUUM_ANALYZE_SCALE_FACTOR  
AUTOVACUUM_ANALYZE_THRESHOLD  
AUTOVACUUM_VACUUM_INSERT_SCALE_FACTOR  
AUTOVACUUM_VACUUM_INSERT_THRESHOLD  
AUTOVACUUM_VACUUM_SCALE_FACTOR  
AUTOVACUUM_VACUUM_THRESHOLD  
AUTOVACUUM_FREEZE_MAX_AGE
```

Sont globaux à la base , et ils interviennent directement dans les règles de déclenchement de l'AUTOVACUUM , ANALYSE et FREEZE des objets de la base.

✚ Pour forcer et maitriser le déclenchement de l'auto VACUUM,ANALYSE et FREEZE il faut jouer sur les valeurs de ces paramètres, comme vu concrètement au chapitre 5.

✚ Il ne faut pas toucher aux paramètres globaux mais configurer ces paramètres pour chaque table vu comme une surcharge.

✚ **RECOMMADATIONS pour augmenter la fréquence de VACUUM_AUTO sur une table avec beaucoup UPDATE ou de DELETE**

Augmenter AUTOVACUUM_VACUUM_SCALE_FACTOR -- pour une augmentation forte.

Augmenter AUTOVACUUM_VACUUM_THRESHOLD -- pour une augmentation faible(tunning)

Exemple pour la table switch

```
ALTER TABLE iidm.switch SET (autovacuum_vacuum_scale_factor = 0.5);
```

RECOMMADATIONS pour augmenter la fréquence de VACUUM_AUTO sur une table avec beaucoup d'INSERT

Augmenter AUTOVACUUM_VACUUM_INSERT_SCALE_FACTOR -- pour une augmentation forte

Augmenter AUTOVACUUM_VACUUM_INSERT_THRESHOLD -- pour une augmentation faible(tunning)

Exemple pour la table switch

```
ALTER TABLE iidm.switch SET (autovacuum_vacuum_insert_scale_factor = 0.5);
```

RECOMMADATIONS pour augmenter la frequence de VACUUM_ANALYSE_AUTO sur une table

Augmenter AUTOVACUUM_ANALYZE_SCALE_FACTOR -- pour une augmentation forte

Augmenter AUTOVACUUM_ANALYZE_THRESHOLD -- pour une augmentation faible(tunning)

Exemple pour la table switch

```
ALTER TABLE iidm.switch SET (autovacuum_analyze_scale_factor = 0.5);
```

-  Les recommandations ci-dessous ne sont qu'en exemple il faudra bien connaitre le comportement des tables en fonction de l'activité de l'application et d'un suivi par (supervision et capacity planning)

7. Pour aller plus loin

-  Si le « tunning » du VACUUM , ANALYZE et FREEZE semble insuffisant pour une grosse charge récurrente, il existe une extension **PG_REPACK** (a installer sur les bases).
-  Pg_repack a l'avantage de pouvoir :

- Être exécuter en pleine production avec un verrou exclusif très court sur les objets (quelques millisecondes).
- De libérer effectivement les lignes mortes dans les blocs de données
- Sans aucun risque de génération de bloc corrompu.
- D'être programmer a heure fixe en fonction de l'activité de la base de Donnée.

FIN DU DOCUMENT